

BACSE203P

Computer Networks Laboratory

Complete Compiled & Annotated Lab Record

This document brings together all lab experiments in chronological order, with plain-language explanations and inline comments added to every program so each script can be understood independently of the others.

#	Experiment	Date
1	Study of Basic Network Commands & Networking Hardware	20 Jul 2026
2	Socket Programming Foundations	22 Jul 2026
3	CRC Error Detection — Generic Sender/Receiver	28 Jul 2026
4	CRC Error Detection — Client/Server over TCP Sockets	28 Jul 2026
5	Internet Checksum — Client/Server Error Detection	04 Aug 2026
6	Hamming Code — Error Detection & Correction (v1)	05 Aug 2026
7	Stop-and-Wait ARQ Flow Control Protocol	17 Aug 2026
8	Hamming Code — Error Detection & Correction (v2, with sample runs)	17 Aug 2026
9	IP Addressing — Client/Server Network Analyzer	01 Sep 2026
10	Remote Procedure Call (RPC) — Arithmetic Server/Client	16 Sep 2026

Experiment 1 — Basic Network Commands & Networking Hardware

Original file: FALLSEM2026-27_..._2026-07-20_Study-of-Basic-Network-Commands.pdf

What this experiment covers

This is a survey / demonstration experiment rather than a program you run. It walks through the everyday command-line tools used to inspect, diagnose, monitor, and secure a network connection, and then shows pictures of the physical hardware (hub, switch, router, access point, NIC, repeater) that those commands are talking to.

Group 1 — Connectivity & Interfaces

ifconfig / **ip addr show** — print each network interface's IP address, MAC address and subnet mask (**ip addr** is the modern replacement for the older **ifconfig**). **ip link show** — shows whether an interface is UP/DOWN and its MTU. **ip route** / **route -n** — print the routing table, i.e. which gateway is used to reach which network. **hostname -I** is a one-line shortcut to just print your IPs.

Group 2 — Basic Diagnostics

ping sends ICMP echo requests to test whether a host is reachable and measures round-trip time. **traceroute** shows every router (hop) a packet passes through on its way to a destination. **nslookup** / **dig** query DNS servers to resolve a domain name to an IP address (**dig** gives more detail). **arp -a** shows the ARP table — the mapping from IP addresses to MAC addresses on your local network.

Group 3 — Advanced Diagnostics

mtr combines ping and traceroute into one continuously-updating view. **whois** looks up who owns/registered a domain. **curl -I** fetches only the HTTP response headers of a web page (fast way to check a server is alive). **wget** downloads a file over HTTP/HTTPS/FTP. **telnet host port** opens a raw TCP connection to test whether a specific port/service is open.

Group 4 — Monitoring

ss -tuln (and the older **netstat -tuln**) list which ports are currently listening for TCP/UDP connections — useful for seeing what services are running. **iftop**, **nload**, **vnstat**, **bmon**, **iptraf**, **sar -n DEV 1**, **dstat -n** all show live bandwidth/traffic statistics in different formats.

Group 5 — Security

tcpdump / **tshark** / **wireshark** capture and inspect raw packets on the wire (Wireshark is the GUI version of tshark). **nmap** scans a host/network to discover which ports and services are open. **nc** (netcat) can open, listen on, or test TCP/UDP connections — often called the "Swiss-army knife" of networking. **ufw status** / **iptables -L** show the firewall's current rules. **scp** securely copies files between machines over SSH.

Well-known port numbers (memorize these — commonly asked)

Protocol	Port(s)	Function
FTP	20, 21	File transfer
SSH	22	Secure remote login / file transfer
Telnet	23	Remote login (insecure, obsolete)
SMTP	25	Sending email
DNS	53	Resolving domain names to IPs
DHCP	67, 68	Dynamic IP address assignment

HTTP	80	Unsecured web traffic
POP3	110	Receiving email (older)
IMAP	143	Receiving email (modern)
HTTPS	443	Encrypted web traffic
MySQL	3306	Database connections
RDP	3389	Windows Remote Desktop
HTTP (alt)	8080	Alternative web traffic port
ICMP	no port	Ping / traceroute / error reporting

Networking Hardware — quick reference

Device	What it does	Key idea
Hub	Repeats incoming data to *every* connected port	No intelligence — broadcasts to all, wastes bandwidth
Switch	Learns MAC addresses, forwards data only to the correct device	Operates at Layer 2 (Data Link)
Router	Connects separate networks (e.g. LAN to Internet)	Operates at Layer 3 (Network) Assigns IP addresses, NAT
Access Point (AP)	Lets wireless devices join a wired network	Bridges Wi-Fi to Ethernet
NIC	Hardware inside a computer that physically connects to a network	Every wired/wireless interface needs one
Repeater	Receives a weak signal and re-transmits it at full strength	Extends the physical range of a network

Note: Layer order to remember: Hub/Repeater/NIC work at the Physical layer, Switch/AP work at the Data Link layer, and Router works at the Network layer.

Experiment 2 — Socket Programming Foundations

Original file: FALLSEM2026-27_..._2026-07-22_Socket-Programming-Foundation.pdf

Concept

A **socket** is an endpoint for sending/receiving data across a network. Python's built-in socket module lets one program (the server) wait for connections and another program (the client) connect to it. This experiment covers the three foundational patterns every later experiment builds on: a basic TCP client-server, a UDP (datagram) client-server, and a TCP client-server with proper exception handling.

1. TCP Client-Server (Stream Socket)

TCP is *connection-oriented*: the client must connect() before any data can flow, and the server must listen() and accept() that connection first. This guarantees reliable, ordered delivery.

```
# ----- SERVER -----
import socket

server_socket = socket.socket()          # create a default TCP socket
server_socket.bind(('localhost', 8080))  # reserve this address:port
server_socket.listen(1)                  # start listening, queue = 1 pending client
print("Server listening...")

conn, addr = server_socket.accept()      # BLOCKS until a client connects
print("Connected to:", addr)             # addr = (client_ip, client_port)

data = conn.recv(1024).decode()          # read up to 1024 bytes, decode bytes->str
print("Received:", data)

conn.send("Message received".encode())   # encode str->bytes before sending
conn.close()                             # close this client's connection
server_socket.close()                   # close the listening socket

# ----- CLIENT -----
import socket

client_socket = socket.socket()
client_socket.connect(('localhost', 8080)) # actively connect to the server

client_socket.send("Hello Server!".encode())
data = client_socket.recv(1024).decode()
print("Server says:", data)

client_socket.close()
```

2. UDP Client-Server (Datagram Socket)

UDP is *connectionless*: there is no connect()/accept() handshake. Each packet ("datagram") is sent independently with sendto()/recvfrom(), which also returns the sender's address each time. UDP is faster but does not guarantee delivery or ordering — used where speed matters more than reliability (e.g. video streaming, DNS).

```
# ----- UDP SERVER -----
import socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) # SOCK_DGRAM = UDP
server_socket.bind(('localhost', 8080))
print("UDP Server ready...")

while True:
    data, addr = server_socket.recvfrom(1024) # no accept() needed for UDP
    print("Received from:", addr, "Message:", data.decode())
    server_socket.sendto("Message received".encode(), addr) # reply straight to that address

# ----- UDP CLIENT -----
```

```

import socket

client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
message = "Hello UDP Server!"
client_socket.sendto(message.encode(), ('localhost', 8080)) # send without connecting first
data, server = client_socket.recvfrom(1024)
print("Server says:", data.decode())
client_socket.close()

```

3. TCP Client-Server with Exception Handling

Real programs must handle failures gracefully: the port may already be in use, the server might not be running when the client tries to connect, etc. Wrapping the socket calls in try / except / finally ensures sockets are always closed cleanly even when something goes wrong.

```

# ----- SERVER -----
import socket
server = None
client = None
try:
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind(("127.0.0.1", 5000))
    server.listen(1)
    print("Server started...")
    print("Waiting for client connection...")

    client, address = server.accept()
    print("Connected to:", address)

    message = client.recv(1024).decode()
    print("Client:", message)

    reply = input("Enter reply: ")
    client.send(reply.encode())

except socket.error as err:
    print("Socket Error:", err) # e.g. "Address already in use"
except Exception as e:
    print("Error:", e)
finally:
    # finally ALWAYS runs, even after an exception, so sockets never leak
    if client:
        client.close()
    if server:
        server.close()
    print("Server Closed.")

# ----- CLIENT -----
import socket
client = None
try:
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect(("127.0.0.1", 5000))

    message = input("Enter message: ")
    client.send(message.encode())

    reply = client.recv(1024).decode()
    print("Server:", reply)

except ConnectionRefusedError:
    print("Server is not running.") # thrown if nothing is listening on that port
except socket.error as err:
    print("Socket Error:", err)
except Exception as e:
    print("Error:", e)
finally:
    if client:
        client.close()

```

```
print("Client Closed.")
```

Bonus: Simple Date & Time Service

A tiny real-world use of the pattern above: the server fetches the current system time and sends it to any client that connects — a miniature version of the NTP (Network Time Protocol) idea.

```
# ----- SERVER -----
import socket
from datetime import datetime          # gives access to the current date/time

server = None
client = None
try:
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # AF_INET=IPv4, SOCK_STREAM=TCP
    server.bind(("127.0.0.1", 5000))
    server.listen(1)          # allow 1 pending connection
    print("DATE & TIME SERVER STARTED")
    print("Waiting for client connection...\n")

    client, address = server.accept() # returns a NEW socket for this client + its address
    print("Client Connected Successfully")
    print("Client Address :", address)

    # %d=day %m=month %Y=year %H=hour(24h) %M=minute %S=second
    current_datetime = datetime.now().strftime("%d-%m-%Y %H:%M:%S")
    print("\nCurrent Date & Time :", current_datetime)

    client.send(current_datetime.encode()) # encode() turns the string into bytes
    print("Date & Time sent successfully.")

except Exception as e:
    print("Error :", e)
finally:
    if client:
        client.close()
    if server:
        server.close()
    print("\nServer Closed.")

# ----- CLIENT -----
import socket
client = None
try:
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect(("127.0.0.1", 5000))
    print("Connected to Date & Time Server\n")

    current_datetime = client.recv(1024).decode() # decode() turns bytes back into a string
    print("Current Date & Time Received from Server")
    print("-----")
    print(current_datetime)

except Exception as e:
    print("Error :", e)
finally:
    if client:
        client.close()
    print("\nConnection Closed.")
```



```

print("\n----- Sender Output -----")
print("Original Data :", data)
print("CRC Remainder :", remainder)
print("Codeword Sent :", codeword)

```

RECEIVER — verifies the codeword

Objective: Verify whether the received codeword contains errors
by dividing it again with the SAME divisor used at the sender.

```

def xor(divisor, dividend):
    result = ""
    for i in range(1, len(dividend)):
        if divisor[i] == dividend[i]:
            result += "0"
        else:
            result += "1"
    return result

def crc_check(received_data, divisor):
    n = len(divisor)
    temp = received_data[:n]

    while n < len(received_data):
        if temp[0] == '1':
            temp = xor(divisor, temp) + received_data[n]
        else:
            temp = xor("0" * len(divisor), temp) + received_data[n]
        n += 1

    if temp[0] == '1':
        temp = xor(divisor, temp)
    else:
        temp = xor("0" * len(divisor), temp)

    return temp    # final remainder

# ----- MAIN PROGRAM -----
print("===== CRC RECEIVER =====\n")

received = input("Enter Received Codeword : ")
divisor = input("Enter Divisor          : ")

remainder = crc_check(received, divisor)

print("\n----- Receiver Output -----")
print("Received Codeword :", received)
print("Remainder          :", remainder)

# If every bit of the remainder is '0', no error was introduced in transit
if set(remainder) == {"0"}:
    print("\nResult : No Error Detected")
else:
    print("\nResult : Error Detected")

# Note: Run order is --
# 1. Run sender -> generates & prints the codeword
# 2. Run receiver -> paste that codeword in as input

```

Sample run

```

SENDER (Server)
-----
Enter Data Bits : 1001
Enter Divisor   : 1101

Data after appending zeros : 1001000

----- Sender Output -----

```

Original Data : 1001
CRC Remainder : 011
Codeword Sent : 1001011

RECEIVER (client) -- codeword received correctly

Enter Received Codeword : 1001011
Enter Divisor : 1101

----- Receiver Output -----
Received Codeword : 1001011
Remainder : 000
Result : No Error Detected

RECEIVER (client) -- codeword corrupted in transit

Enter Received Codeword : 1001000
Enter Divisor : 1101

----- Receiver Output -----
Received Codeword : 1001000
Remainder : 011
Result : Error Detected

Experiment 4 — CRC Error Detection over TCP Sockets

Original file: FALLSEM2026-27_..._2026-07-28_CRC-server-client-.pdf

Concept

This experiment takes the CRC logic from Experiment 3 and puts it on either end of a real TCP connection: the **client acts as the sender** (computes the CRC and transmits the codeword), and the **server acts as the receiver** (recomputes the CRC on what it received). If the remainder is all zeros, the transmission is considered error-free. The socket here is purely the transport channel — the interesting logic is identical to Experiment 3's Modulo-2 division.

CRC SERVER (Receiver)

```
import socket

# Modulo-2 (XOR) division, used to verify the received codeword
def crc_check(data, divisor):
    data = list(data)
    n = len(divisor)
    for i in range(len(data) - n + 1):
        if data[i] == '1':
            for j in range(n):
                data[i + j] = str(int(data[i + j]) ^ int(divisor[j])) # XOR bit by bit
    return ''.join(data[-(n - 1):]) # last (n-1) bits = the remainder

HOST = "127.0.0.1"
PORT = 5000

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1) # lets the port be reused
                                                                # immediately after restart
server.bind((HOST, PORT))
server.listen(1)
print("=====")
print("    CRC SERVER STARTED")
print("=====")
print("Waiting for Client...\n")

conn, addr = server.accept()
print("Connected by :", addr)

message = conn.recv(1024).decode()
codeword, divisor = message.split("|") # sender packs "codeword|divisor" together

print("\nReceived Codeword :", codeword)
print("Generator      :", divisor)

remainder = crc_check(codeword, divisor)
print("Calculated CRC  :", remainder)

if remainder == "0" * (len(divisor) - 1):
    print("\nResult : No Error Detected")
else:
    print("\nResult : Error Detected")

conn.close()
server.close()
```

CRC CLIENT (Sender)

```
import socket

# Generate the CRC codeword for the data using the given divisor
def crc_generate(data, divisor):
    appended = data + "0" * (len(divisor) - 1) # pad with (len(divisor)-1) zeros
    dividend = list(appended)
```

```

n = len(divisor)

for i in range(len(data)):
    if dividend[i] == '1':
        for j in range(n):
            dividend[i + j] = str(int(dividend[i + j]) ^ int(divisor[j]))

remainder = ''.join(dividend[-(n - 1):])
codeword = data + remainder
return codeword, remainder

HOST = "127.0.0.1"
PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

print("=====")
print("    CRC CLIENT")
print("=====\\n")

data = input("Enter Data Bits : ")
divisor = input("Enter Divisor   : ")

codeword, remainder = crc_generate(data, divisor)

print("\\nCRC Remainder :", remainder)
print("Codeword      :", codeword)

# Pack both the codeword AND the divisor into one message, separated by '|',
# so the server on the other end knows which divisor to check against.
message = codeword + "|" + divisor
client.send(message.encode())

print("\\nCodeword Sent Successfully.")
client.close()

```

Roles at a glance

Client (Sender)	Server (Receiver)
Creates TCP socket	Creates TCP socket
Connects to server	Waits for connection
Generates CRC	Receives codeword
Sends codeword	Checks CRC
Closes socket	Displays result and closes socket

Note: Run order: start the server first, then run the client. Sample run: data=1001, divisor=1101 -> codeword 1001011 -> "No Error Detected" at the server.

Experiment 5 — Internet Checksum Error Detection

Original file: FALLSEM2026-27_..._2026-08-04_Internet-checksum-Error-detection.pdf

Concept

The **Internet checksum** (used in IP, TCP, UDP headers) is a simpler, faster alternative to CRC. The message is split into 16-bit words; all the words are added together with *carry wrap-around* (any overflow past 16 bits is added back into the low 16 bits); finally the whole sum is bitwise complemented (~). The sender transmits the message plus this checksum. The receiver repeats the same calculation on the message it received and compares its own checksum against the one it was sent.

Flow: Client enters a message → computes checksum → sends "message + checksum" to the server. Server receives it → recalculates the checksum independently → compares the two → reports Checksum VALID or INVALID.

SERVER

```
import socket

# Function to calculate the Internet checksum of a text message
def checksum(data):
    if len(data) % 2 != 0:
        data += " "          # pad with a space so we always have whole 16-bit (2-char) words

    total = 0
    for i in range(0, len(data), 2):
        # combine two characters into one 16-bit "word": first char -> high byte
        word = (ord(data[i]) << 8) + ord(data[i + 1])
        total += word
        while total > 0xFFFF:          # if the sum overflows 16 bits...
            total = (total & 0xFFFF) + (total >> 16) # ...wrap the overflow back around (
carry)

    return (~total) & 0xFFFF          # one's complement of the final sum, masked to 16 bits

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("localhost", 5000))
server.listen(1)
print("Server Waiting...")

conn, addr = server.accept()
data = conn.recv(1024).decode()

# Message format sent by client: "message|checksum_in_hex"
message, received_checksum = data.split("|")
received_checksum = int(received_checksum, 16)          # parse the hex string back to an int

calculated_checksum = checksum(message)                # recompute independently

print("\nMessage Received :", message)
print("Received Checksum :", format(received_checksum, "04X"))
print("Calculated Checksum:", format(calculated_checksum, "04X"))

if received_checksum == calculated_checksum:
    print("Checksum VALID")
    conn.send("Checksum VALID".encode())
else:
    print("Checksum INVALID")
    conn.send("Checksum INVALID".encode())

conn.close()
server.close()
```

CLIENT

```

import socket

# Identical checksum function -- MUST match the server's implementation exactly
def checksum(data):
    if len(data) % 2 != 0:
        data += " "
    total = 0
    for i in range(0, len(data), 2):
        word = (ord(data[i]) << 8) + ord(data[i + 1])
        total += word
        while total > 0xFFFF:
            total = (total & 0xFFFF) + (total >> 16)
    return (~total) & 0xFFFF

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("localhost", 5000))

message = input("Enter Message : ")
chk = checksum(message)
print("Checksum Generated :", format(chk, "04X"))      # 04X = 4-digit uppercase hex, zero-
padded

packet = message + "|" + format(chk, "04X")
client.send(packet.encode())

reply = client.recv(1024).decode()
print(reply)      # prints "Checksum VALID" or "Checksum INVALID" from the server

client.close()

```

Note: Because the checksum is computed identically on data that hasn't changed, an unmodified message will always come back VALID. To see INVALID, the message text would need to differ between what was checksummed and what was actually received (e.g. simulate corruption by editing the packet in transit).

Experiment 6 — Hamming Code Error Detection & Correction (v1)

Original file: FALLSEM2026-27_..._2026-08-05_Hamming-Code-Error-Detection-.pdf

Concept

Unlike CRC/checksum, which only *detect* errors, **Hamming Code** can also *locate and correct* a single-bit error. Extra **parity bits** are inserted at positions that are powers of two (1, 2, 4, 8, ...) within the data. Each parity bit covers a specific subset of bit positions (determined by binary representation), so that if one bit flips during transmission, XOR-ing the parity checks together at the receiver reconstructs the exact bit position of the error (called the **syndrome**).

Here the **sender is the server**: it builds the Hamming code and (optionally) flips one bit to simulate a transmission error. The **receiver is the client**: it recalculates the parity/syndrome, reports the error position (0 = no error), and corrects that single bit if needed.

SERVER (Sender)

```
import socket

# How many redundant (parity) bits 'r' are needed for 'm' data bits,
# such that  $2^r \geq m + r + 1$ 
def calc_redundant_bits(m):
    r = 0
    while (2 ** r) < (m + r + 1):
        r += 1
    return r

# Interleave the data bits with placeholder '0's at every power-of-two position
# (1, 2, 4, 8, ...) -- these placeholders will later hold the actual parity bits
def insert_parity_bits(data, r):
    j = 0
    k = 0
    m = len(data)
    res = []
    for i in range(1, m + r + 1):
        if i == 2 ** j:
            res.append('0')    # reserve this position for a parity bit
            j += 1
        else:
            res.append(data[m - 1 - k])    # fill in the next data bit
            k += 1
    return res[::-1]    # reverse -> bit 1 (LSB) ends up first in processing order

# Compute the actual value of each parity bit by XOR-ing the bits it covers
def calculate_parity(code, r):
    n = len(code)
    for i in range(r):
        parity = 0
        p = 2 ** i    # this parity bit's position: 1, 2, 4, 8...
        for j in range(1, n + 1):
            if j & p:    # does position j fall under parity bit p's coverage?
                parity ^= int(code[-j])
        code[-p] = str(parity)
    return code

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
try:
    server.bind(("127.0.0.1", 5000))
    server.listen(1)
    print("Server Waiting...")

    conn, addr = server.accept()
    print("Connected by:", addr)
```

```

data = input("Enter Binary Data: ")
if not all(bit in '01' for bit in data):
    raise ValueError("Binary digits only.")

r = calc_redundant_bits(len(data))
code = insert_parity_bits(data, r)
code = calculate_parity(code, r)
hamming = "".join(code)
print("Generated Hamming Code:", hamming)

# Optionally corrupt one bit, to test that the client can detect & fix it
ch = input("Introduce transmission error (y/n)? ")
if ch.lower() == 'y':
    pos = int(input("Enter error position (from right): "))
    if pos < 1 or pos > len(hamming):
        raise ValueError("Invalid position.")
    temp = list(hamming)
    index = len(temp) - pos
    temp[index] = '1' if temp[index] == '0' else '0'    # flip that single bit
    hamming = "".join(temp)
    print("Code Sent:", hamming)

conn.send(hamming.encode())

except Exception as e:
    print("Error:", e)
finally:
    try:
        conn.close()
    except:
        pass
    server.close()
    print("Server Closed.")

```

CLIENT (Receiver)

```

import socket

# Recompute each parity check on the received code; any parity that comes out
# non-zero contributes its position value to 'error' -- the sum pinpoints
# the exact (1-indexed, from the right) position of the flipped bit.
def detect_error(code, r):
    n = len(code)
    error = 0
    for i in range(r):
        parity = 0
        p = 2 ** i
        for j in range(1, n + 1):
            if j & p:
                parity ^= int(code[-j])
        if parity:
            error += p
    return error

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
try:
    print("Connecting...")
    client.connect(("127.0.0.1", 5000))

    code = client.recv(1024).decode()
    print("\nReceived Code :", code)

    r = 0
    while (2 ** r) < (len(code) + 1):
        r += 1

    error = detect_error(code, r)

    if error == 0:
        print("No Error Detected.")

```

```

else:
    print("Error Detected at Position:", error)
    code = list(code)
    index = len(code) - error
    code[index] = '1' if code[index] == '0' else '0'    # flip the bad bit back
    corrected = "".join(code)
    print("Corrected Code:", corrected)

except ConnectionRefusedError:
    print("Server is not running.")
except Exception as e:
    print("Error:", e)
finally:
    client.close()
    print("Client Closed.")

```

Sample run

SERVER

```

-----
Server Waiting...
Connected by: ('127.0.0.1', 57786)
Enter Binary Data: 1100110
Generated Hamming Code: 11000110010
Introduce transmission error (y/n)? y
Enter error position (from right): 4
Code Sent: 11000111010
Server Closed.

```

CLIENT

```

-----
Connecting...
Received Code : 11000111010
Error Detected at Position: 4
Corrected Code: 11000110010
Client Closed.

```

Experiment 7 — Stop-and-Wait ARQ Flow Control

Original file: FALLSEM2026-27_..._2026-08-17_Flow-Control---Stop-N-Wait.pdf

Concept

Stop-and-Wait is the simplest data-link flow-control / ARQ (Automatic Repeat reQuest) protocol: the sender transmits exactly **one frame**, then stops and waits for an acknowledgment (ACK) before sending the next one. To detect duplicate frames (e.g. if an ACK is lost and the sender retransmits), each frame carries a single **sequence number that alternates between 0 and 1**. If the ACK doesn't arrive (or the wrong one arrives), the sender retransmits the same frame.

RECEIVER

```
import socket
HOST = "localhost"
PORT = 5000
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)

print("Receiver started...")
print("Waiting for Sender connection...")
conn, addr = server.accept()
print("Sender connected:", addr)

expected_sequence = 0          # the receiver only accepts frames with this sequence number

while True:
    data = conn.recv(1024)
    if not data:
        break
    message = data.decode()

    if message == "END":          # sender signals it is done
        print("\nAll frames received.")
        print("Receiver terminating...")
        break

    # message format: "sequence:frame_data"
    sequence, frame = message.split(":", 1)
    sequence = int(sequence)

    print("\n-----")
    print("Receiver")
    print("-----")
    print("Frame received :", frame)
    print("Sequence number:", sequence)

    if sequence == expected_sequence:
        # correct, in-order frame
        print("Frame accepted")
        ack = "ACK:" + str(sequence)
        conn.send(ack.encode())
        print("ACK sent      :", ack)
        expected_sequence = 1 - expected_sequence  # flip 0<->1 for the next expected frame
    else:
        # this is a duplicate of the PREVIOUS frame (its ACK must have been lost) --
        # re-send that previous ACK instead of accepting the frame again
        print("Duplicate frame received")
        previous_sequence = 1 - expected_sequence
        ack = "ACK:" + str(previous_sequence)
        conn.send(ack.encode())
        print("Duplicate ACK sent:", ack)

conn.close()
```

```
server.close()
print("\nReceiver terminated.")
```

SENDER

```
import socket
HOST = "localhost"
PORT = 5000
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))
print("Connected to Receiver")

frames = []
n = int(input("Enter number of frames: "))
for i in range(n):
    data = input("Enter data for Frame " + str(i) + ": ")
    frames.append(data)

sequence = 0
for i in range(len(frames)):
    acknowledged = False

    while not acknowledged:          # keep retrying until the correct ACK comes back
        frame = frames[i]
        message = str(sequence) + ":" + frame
        print("\n-----")
        print("Sender")
        print("-----")
        print("Sending Frame :", frame)
        print("Sequence No   :", sequence)

        client.send(message.encode())
        ack = client.recv(1024).decode()
        print("ACK received  :", ack)

        ack_type, ack_number = ack.split(":")
        ack_number = int(ack_number)

        if ack_number == sequence:
            print("Frame successfully transmitted")
            acknowledged = True          # move on to the next frame
        else:
            print("Wrong ACK received")
            print("Retransmitting frame...") # stays in the while loop, resends same frame

    sequence = 1 - sequence            # toggle sequence number 0 <-> 1

client.send("END".encode())           # tell the receiver there is nothing more coming
client.close()
print("\nAll frames transmitted successfully.")
```

Sample run (2 frames, no loss)

RECEIVER	SENDER
-----	-----
Receiver started...	Connected to Receiver
Waiting for Sender connection...	Enter number of frames: 2
Sender connected: ('127.0.0.1', 59353)	Enter data for Frame 0: hi
	Enter data for Frame 1: hi

Receiver	
-----	-----
Frame received : hi	Sender
Sequence number: 0	-----
Frame accepted	
ACK sent : ACK:0	Sending Frame : hi
	Sequence No : 0
	ACK received : ACK:0
	Frame successfully transmitted

Frame received : hi
Sequence number: 1
Frame accepted
ACK sent : ACK:1

Sending Frame : hi
Sequence No : 1
ACK received : ACK:1
Frame successfully transmitted

All frames received.
Receiver terminating...

Note: Run order matters: start the RECEIVER first (it must be listening before the sender tries to connect), then start the SENDER.

Experiment 8 — Hamming Code Error Detection & Correction (v2)

Original file: FALLSEM2026-27_..._2026-08-17_Hamming-Code---error-detection.pdf

What's different from Experiment 6

This is a repeat run of the same Hamming Code sender/receiver program from Experiment 6 (identical algorithm — `calc_redundant_bits`, `insert_parity_bits`, `calculate_parity` on the server; `detect_error` on the client). It is included here to keep the lab record complete and chronological; refer back to Experiment 6 for the fully-commented code listing. Only the captured sample output differs slightly below.

Sample run captured for this submission

```
SERVER (run 1 -- no error introduced)
-----
Server Waiting...
Connected by: ('127.0.0.1', 57785)
Enter Binary Data: 1100110
Generated Hamming Code: 11000110010
Introduce transmission error (y/n)? n
Server Closed.

SERVER (run 2 -- error introduced at position 4)
-----
Server Waiting...
Connected by: ('127.0.0.1', 57786)
Enter Binary Data: 1100110
Generated Hamming Code: 11000110010
Introduce transmission error (y/n)? y
Enter error position (from right): 4
Code Sent: 11000111010
Server Closed.

CLIENT (run 1 -- matches "no error" server run)
-----
Connecting...
Received Code : 11000110010
No Error Detected.
Client Closed.

CLIENT (run 2 -- matches "error introduced" server run)
-----
Connecting...
Received Code : 11000111010
Error Detected at Position: 4
Corrected Code: 11000110010
Client Closed.
```

Note: Notice the corrected code (11000110010) exactly matches the originally generated code before the deliberate 1-bit corruption — this confirms the Hamming code successfully located and fixed the injected error.

Experiment 9 — IP Addressing: Client/Server Network Analyzer

Original file: FALLSEM2026-27_..._2026-09-01_ip-addressing.pdf

Concept

This experiment builds a small interactive TCP server that offers several IPv4-related utilities to any connected client, using Python's `ipaddress` module: classifying an address into Class A/B/C/D/E, subnetting (network/broadcast/host range) given a CIDR prefix, checking whether two IPs fall in the same subnet, and converting an IP to its 32-bit binary form. It also demonstrates **multi-client support using threading** — each connecting client is handled on its own thread so multiple clients can be served at the same time.

SERVER — helper functions

```
import socket
import threading
import ipaddress

HOST = "0.0.0.0"      # listen on all available network interfaces
PORT = 5000

# Discover this machine's own LAN IP by opening a dummy UDP "connection"
# to a public address (no actual packets are sent for UDP connect())
def get_server_ip():
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.connect(("8.8.8.8", 80))
        ip = s.getsockname()[0]      # the local IP the OS would use for that route
        s.close()
        return ip
    except Exception:
        return "127.0.0.1"

# Classify an IPv4 address by its first octet, and report several properties
def analyze_ip(ip):
    try:
        address = ipaddress.IPv4Address(ip)      # validates + parses the address
        first_octet = int(ip.split('.')[0])

        if 1 <= first_octet <= 126:
            ip_class = "Class A"; default_mask = "255.0.0.0"
        elif 128 <= first_octet <= 191:
            ip_class = "Class B"; default_mask = "255.255.0.0"
        elif 192 <= first_octet <= 223:
            ip_class = "Class C"; default_mask = "255.255.255.0"
        elif 224 <= first_octet <= 239:
            ip_class = "Class D (Multicast)"; default_mask = "N/A"
        else:
            ip_class = "Class E (Experimental)"; default_mask = "N/A"

        binary = format(int(address), "032b")      # 32-bit binary string
        binary = ".".join(binary[i:i+8] for i in range(0, 32, 8))      # group into 4 octets

        return f"""...IPv4 ANALYSIS...
IP Address      : {address}
IP Class       : {ip_class}
Default Mask    : {default_mask}
Private Address: {address.is_private}
Loopback Address: {address.is_loopback}
Multicast      : {address.is_multicast}
Global Address : {address.is_global}
Reserved       : {address.is_reserved}
Binary Address  : {binary}"""
    except ValueError:
        return "ERROR: Invalid IPv4 address."
```

```

# Given an IP and CIDR prefix (e.g. 192.168.1.7/24), compute the whole subnet
def subnet_analysis(ip, prefix):
    try:
        network = ipaddress.ip_network(f"{ip}/{prefix}", strict=False)
        total = network.num_addresses
        usable = total - 2 if total >= 2 else 0    # subtract network & broadcast addresses
        hosts = list(network.hosts())
        first_host = hosts[0] if hosts else "N/A"
        last_host = hosts[-1] if hosts else "N/A"
        return f"""...SUBNET ANALYSIS...
Subnet Mask      : {network.netmask}
Network Address: {network.network_address}
Broadcast       : {network.broadcast_address}
First Host      : {first_host}
Last Host       : {last_host}
Total Addresses: {total}
Usable Hosts    : {usable}"""
    except ValueError:
        return "ERROR: Invalid IP address or CIDR prefix."

# Check whether two IPs belong to the same subnet under a given prefix
def same_network(ip1, ip2, prefix):
    try:
        network1 = ipaddress.ip_network(f"{ip1}/{prefix}", strict=False)
        network2 = ipaddress.ip_network(f"{ip2}/{prefix}", strict=False)
        if network1.network_address == network2.network_address:
            return "Result : SAME NETWORK ..."
        else:
            return "Result : DIFFERENT NETWORKS ..."
    except ValueError:
        return "ERROR: Invalid IP address or prefix."

# Convert a dotted-decimal IPv4 address into its 32-bit binary representation
def ip_to_binary(ip):
    try:
        address = ipaddress.IPv4Address(ip)
        binary = format(int(address), "032b")
        formatted = ".".join(binary[i:i+8] for i in range(0, 32, 8))
        return f"IP Address : {address}\nBinary : {formatted}"
    except ValueError:
        return "ERROR: Invalid IPv4 address."

```

SERVER — per-client menu handler + main loop

```

def send_menu(client):
    menu = """
IP ADDRESSING SERVER
1. Analyze IPv4 Address
2. Subnet Analysis
3. Check Same Network
4. IP Address to Binary
5. Client IP Information
6. Exit
Enter your choice:
"""
    client.sendall(menu.encode())

# Runs in its OWN THREAD for every connected client (see threading.Thread below),
# so multiple clients can use the server's menu at the same time without blocking
# each other.
def handle_client(client, address):
    print(f"[+] Client connected: {address}")
    try:
        while True:
            send_menu(client)
            choice = client.recv(1024).decode().strip()
            if not choice:
                break

            if choice == "1":

```

```

        client.sendall(b"Enter IPv4 address: ")
        ip = client.recv(1024).decode().strip()
        result = analyze_ip(ip)
        client.sendall(result.encode())

    elif choice == "2":
        client.sendall(b"Enter IPv4 address: ")
        ip = client.recv(1024).decode().strip()
        client.sendall(b"Enter CIDR prefix (example: 24): ")
        prefix = client.recv(1024).decode().strip()
        try:
            prefix = int(prefix)
            result = subnet_analysis(ip, prefix)
        except ValueError:
            result = "ERROR: Prefix must be an integer."
        client.sendall(result.encode())

    elif choice == "3":
        client.sendall(b"Enter first IPv4 address: ")
        ip1 = client.recv(1024).decode().strip()
        client.sendall(b"Enter second IPv4 address: ")
        ip2 = client.recv(1024).decode().strip()
        client.sendall(b"Enter CIDR prefix (example: 24): ")
        prefix = client.recv(1024).decode().strip()
        try:
            prefix = int(prefix)
            result = same_network(ip1, ip2, prefix)
        except ValueError:
            result = "ERROR: Prefix must be an integer."
        client.sendall(result.encode())

    elif choice == "4":
        client.sendall(b"Enter IPv4 address: ")
        ip = client.recv(1024).decode().strip()
        result = ip_to_binary(ip)
        client.sendall(result.encode())

    elif choice == "5":
        # address = (client_ip, client_port), captured by server.accept()
        client_ip, client_port = address[0], address[1]
        result = f"CLIENT INFORMATION\nClient IP Address : {client_ip}" \
            f"\nClient Port      : {client_port}"
        client.sendall(result.encode())

    elif choice == "6":
        client.sendall(b"Connection closed. Thank you!\n")
        break

    else:
        client.sendall(b"ERROR: Invalid menu choice.\n")

except ConnectionResetError:
    print(f"[-] Client disconnected: {address}")
except Exception as e:
    print(f"[!] Error: {e}")
finally:
    client.close()
    print(f"[-] Connection closed: {address}")

def main():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1) # allow quick restart
    server.bind((HOST, PORT))
    server.listen(5) # up to 5 pending connections in the queue
    server_ip = get_server_ip()

    print("IP ADDRESSING SERVER")
    print("Server IP :", server_ip)
    print("Port      :", PORT)

```

```

print("Protocol : TCP")
print("Status : Waiting for clients...")

try:
    while True:
        client, address = server.accept()
        # hand this client off to its own thread so the main loop can keep
        # accepting new clients immediately -- this is what enables multiple
        # simultaneous connections
        thread = threading.Thread(target=handle_client, args=(client, address),
                                   daemon=True)

        thread.start()
except KeyboardInterrupt:
    print("\nServer shutting down...")
finally:
    server.close()

if __name__ == "__main__":
    main()

```

CLIENT

```

import socket
PORT = 5000

def main():
    server_ip = input("Enter Server IP address: ").strip()
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    try:
        client.connect((server_ip, PORT))
        print("\n=====")
        print(" Connected to IP Addressing Server")
        print("=====")

        while True:
            menu = client.recv(4096).decode()          # server-side menu text
            print(menu, end="")
            choice = input().strip()
            client.sendall(choice.encode())

            if choice == "1":
                print(client.recv(1024).decode(), end="")
                ip = input()
                client.sendall(ip.encode())
                print(client.recv(4096).decode())

            elif choice == "2":
                print(client.recv(1024).decode(), end="")
                ip = input()
                client.sendall(ip.encode())
                print(client.recv(1024).decode(), end="")
                prefix = input()
                client.sendall(prefix.encode())
                print(client.recv(4096).decode())

            elif choice == "3":
                print(client.recv(1024).decode(), end="")
                ip1 = input()
                client.sendall(ip1.encode())
                print(client.recv(1024).decode(), end="")
                ip2 = input()
                client.sendall(ip2.encode())
                print(client.recv(1024).decode(), end="")
                prefix = input()
                client.sendall(prefix.encode())
                print(client.recv(4096).decode())

            elif choice == "4":
                print(client.recv(1024).decode(), end="")
                ip = input()

```

```

        client.sendall(ip.encode())
        print(client.recv(4096).decode())

    elif choice == "5":
        print(client.recv(4096).decode())          # server sends client info directly

    elif choice == "6":
        print(client.recv(4096).decode())
        break                                       # exit the menu loop

    else:
        print(client.recv(4096).decode())

except ConnectionRefusedError:
    print("\nERROR: Could not connect to server.")
    print("Check the server IP and port.")
except socket.timeout:
    print("\nERROR: Connection timed out.")
except Exception as e:
    print("\nERROR:", e)
finally:
    client.close()

if __name__ == "__main__":
    main()

```

Sample run

SERVER

```

-----
IP ADDRESSING SERVER
Server IP      : 192.168.1.8
Port          : 5000
Protocol      : TCP
Status        : Waiting for clients...
[+] Client connected: ('192.168.1.8', 61903)

```

CLIENT -- Option 1 (Analyze IPv4 Address)

```

-----
Enter Server IP address: 192.168.1.8
Enter your choice: 1
Enter IPv4 address: 192.168.1.7

IPv4 ANALYSIS
IP Address      : 192.168.1.7
IP Class       : Class C
Default Mask    : 255.255.255.0
Private Address : True
Loopback Address: False
Multicast      : False
Global Address  : False
Reserved       : False
Binary Address  : 11000000.10101000.00000001.00000111

```

CLIENT -- Option 2 (Subnet Analysis) -- invalid prefix example

```

-----
Enter your choice: 2
Enter IPv4 address: 192.168.1.7
Enter CIDR prefix (example: 24): 45
ERROR: Invalid IP address or CIDR prefix.          <- prefixes must be 0-32

```

CLIENT -- Option 3 (Check Same Network)

```

-----
Enter your choice: 3
Enter first IPv4 address : 192.168.1.97
Enter second IPv4 address: 192.168.1.9

```

Enter CIDR prefix (example: 24): 24

NETWORK COMPARISON

Result : SAME NETWORK

Network : 192.168.1.0/24

CLIENT -- Option 5 (Client IP Information) then Option 6 (Exit)

Enter your choice: 5

CLIENT INFORMATION

Client IP Address : 192.168.1.8

Client Port : 61903

Enter your choice: 6

Connection closed. Thank you!

Experiment 10 — Remote Procedure Call (RPC): Arithmetic Server/Client

Original file: FALLSEM2026-27_..._2026-09-16_RPC-program.pdf

Concept

RPC (Remote Procedure Call) lets a client call a function that actually executes on a remote server, as if it were a normal local function call — all the network plumbing (serializing arguments, sending them, waiting for the result) is hidden away by the RPC library. Here that library is Python's built-in xmlrpc module: the server registers plain Python functions (add, subtract, multiply, ...), and the client calls them through a ServerProxy object almost exactly like calling a local function, e.g. client.add(a, b).

Algorithm — Server

1. Import the XML-RPC server module.
2. Define the arithmetic functions.
3. Create an RPC server bound to port 8000.
4. Register every function under a name.
5. Call serve_forever() so the server waits for and executes remote calls indefinitely.

Algorithm — Client

1. Import the XML-RPC client module.
2. Connect to the server via ServerProxy.
3. Show a menu, read the user's choice and two numbers.
4. Invoke the matching remote function (e.g. client.add(a, b)) — this transparently sends the call over the network and returns the server's result.
5. Repeat until the user chooses Exit.

SERVER

```
from xmlrpc.server import SimpleXMLRPCServer

# --- Arithmetic functions that will be exposed to remote clients ---
def add(a, b):      return a + b
def subtract(a, b): return a - b
def multiply(a, b): return a * b

def divide(a, b):
    if b == 0:
        return "Error: Division by zero is not allowed."
    return a / b

def power(a, b):    return a ** b

def modulus(a, b):
    if b == 0:
        return "Error: Modulus by zero is not allowed."
    return a % b

# Create the RPC server, listening on localhost:8000.
# allow_none=True lets functions return Python's None value over XML-RPC
# (XML-RPC doesn't support None by default).
server = SimpleXMLRPCServer(("localhost", 8000), allow_none=True)

print("=====")
print("  RPC SERVER STARTED")
print("Server running on Port 8000")
print("Waiting for client requests...")
print("=====")

# Expose each local function under a public name that the client will call by
server.register_function(add, "add")
server.register_function(subtract, "subtract")
server.register_function(multiply, "multiply")
server.register_function(divide, "divide")
server.register_function(power, "power")
server.register_function(modulus, "modulus")
```

```
server.serve_forever()      # blocks here, handling one remote call after another
```

CLIENT

```
import xmlrpc.client

# ServerProxy is the "magic" object: any attribute you call on it
# (e.g. client.add(...)) is transparently turned into a network request
# to the matching function registered on the server.
client = xmlrpc.client.ServerProxy(
    "http://localhost:8000/",
    allow_none=True
)

while True:
    print("\n===== RPC MENU =====")
    print("1. Addition")
    print("2. Subtraction")
    print("3. Multiplication")
    print("4. Division")
    print("5. Power")
    print("6. Modulus")
    print("7. Exit")
    print("=====")

    choice = int(input("Enter your choice: "))

    if choice == 7:
        print("Exiting Client...")
        break

    a = float(input("Enter First Number : "))
    b = float(input("Enter Second Number: "))

    if choice == 1:
        result = client.add(a, b)          # looks like a local call...
        print("Result =", result)         # ...but actually ran on the server!
    elif choice == 2:
        result = client.subtract(a, b)
        print("Result =", result)
    elif choice == 3:
        result = client.multiply(a, b)
        print("Result =", result)
    elif choice == 4:
        result = client.divide(a, b)
        print("Result =", result)
    elif choice == 5:
        result = client.power(a, b)
        print("Result =", result)
    elif choice == 6:
        result = client.modulus(a, b)
        print("Result =", result)
    else:
        print("Invalid Choice! Please Try Again.")
```

Note: Because XML-RPC serializes every call/response as XML text over HTTP, the arguments and return values must be XML-RPC-friendly types (numbers, strings, lists, dicts, booleans, and None when `allow_none=True`) — this is why the client converts inputs with `float(input(...))` rather than passing raw strings.